



Performance Report

Cloud Computing

Gabriele Granzotto

Università degli studi di Trieste

gabriele.granzotto@studenti.units.it

March 26, 2026

Contents

1	Introduction	3
1.1	Tasks and Goals	3
1.2	Report Structure	3
2	Virtual Machines Setup	4
2.1	Create the VM	4
2.1.1	set VirtualBox	4
2.1.2	install the OS	4
2.1.3	configure the template	5
2.1.4	create nodes	5
2.2	Create the Internal Network	6
2.2.1	attach all nodes	6
2.2.2	master-nodes configuration	7
2.2.3	DNSMASQ	8
2.2.4	remote access: SSH/SCP	10
2.3	Shared Filesystem	10
2.4	Useful Scripts	11
3	Containers Setup	13
3.1	Create Containers Structure	13
3.1.1	docker compose	14
3.1.2	shared filesystem	15
3.1.3	resource limits	16
3.2	Configuration Files	17
3.2.1	master Dockerfile	17
3.2.2	nodes Dockerfile	18
3.3	SSH problems & strategy	19
3.4	Terminal in a Docker Machine	21
4	Performance Tests	22
4.1	Methodology & Applied Tests	22

4.1.1	mpirun	23
4.1.2	HPCC	23
4.1.3	stress-ng	25
4.1.4	sysbench	26
4.1.5	iperf3	26
4.1.6	iozone	27
4.2	Results: Processes	27
4.2.1	HPL	28
4.2.2	FFT	29
4.2.3	starDGEMM	30
4.2.4	stress-ng	30
4.2.5	sysbench	30
4.3	Results: Memory	31
4.3.1	MPI random access	31
4.3.2	starSTREAM triad	31
4.3.3	stress-ng	33
4.3.4	sysbench	33
4.4	Results: Disk	35
4.4.1	iozone: no direct I/O	35
4.4.2	iozone: direct I/O	36
4.5	Results: Communication	39
4.5.1	iperf3	39
5	Conclusions	40

1. Introduction

In the modern computer science industry, Software as a Service (SaaS) products and web applications are becoming increasingly important.

To fulfill this need, the industry creates tools to make developers' jobs easier and improve the scalability factor at the same time.

These two tools are:

- Virtual Machines
- Containers

1.1. Tasks and Goals

The goal of this report is to implement a cluster of Virtual Machines and a cluster of Containers.

Then I have to evaluate and compare the performance in computational power, memory, disk and network speed between the two infrastructures.

1.2. Report Structure

The program that I use to implement the virtual machine is VirtualBox. Instead, for the container, I use Docker.

The report will be structured into three parts:

- a VMs setup section;
- a Containers setup section;
- the analysis of all the tests I make.

2. Virtual Machines Setup

Virtual Machines (VMs) are essential components of modern infrastructure, enabling multiple operating systems to run concurrently on a single physical host. This report analyzes the critical performance metrics of these virtualized environments.

2.1. Create the VM

I decide to use Oracle VirtualBox as a VM program. The reasons are that it is one of the most popular VM programs and most supported too. Nevertheless, it accurately represents the performance of that category of program.

2.1.1. set VirtualBox

The process of creating of a VM is quite easy, the entire process is done via a GUI:

- press *New* button;
- insert *Name* of the VM and upload the ISO file of the chosen OS;
- choose CPU, RAM and DISK usage for the VM.

I choose to create a VM with 2 Core of CPU, 2GB of RAM and 25GB of virtual disk. I think it is important to list the host machine specifications, to better interpret the results of tests that will follow.

CPU:	Intel Core i5-8365U 1.60GHz
RAM:	16GB DDR4
DISK:	256GB SSD NVMe

Table 2.1: Hardware Specification

2.1.2. install the OS

As Operative System, I choose to install **Ubuntu Server 24.04**. The reasons of this decision are the widely diffusion of this OS — giving me the possibility of fixing possible

problems searching solutions online — and because of the stability of this distro in comparison of other like Arch Linux, which has a rolling release frequently that often introduces incompatibility problems between programs or libraries.

Obviously a Linux distribution was mandatory to use standard and well established tests. The last reason why I chose Ubuntu is that it is my host machine OS, so it is very convenient for me.

2.1.3. configure the template

After creating the VM, it is important to install the OS and configure it properly.

As the first step is important to ensure that the default Network Connection protocol is **NAT**, the machine will be automatically assigned to an internal IP and will be able to access the Internet.

When the installation procedure asks for which percentage of disk is to be used for installation, it is important to allocate all the space to the OS.

After the insertion of name of your user, password and enable open-ssh I shutdown the machine and then restart.

Code 1: Shutdown in the Terminal

```
sudo shutdown -h now
```

It is important to update the OS and do the upgrade.

Code 2: Update & Upgrade

```
sudo apt update -y && sudo apt upgrade -y
```

We now need to install some crucial programs.

Code 3: Useful Programs

```
sudo apt install net-tool  
sudo apt install gcc cmake
```

2.1.4. create nodes

After all these steps, the VM is ready and we need to create more than one machine to create our cluster.

I decided to use my machine as the **Master** and use the *clone* option twice to create my two **Nodes** (node01 & node02).

2.2. Create the Internal Network

Now it is important to create a **Network** that allows all nodes to communicate with each other. To do so in *Tool* $\rightarrow$ *Network*, there is a panel called *Host-only Networks*, there I created a new network and I call it **CloudBasicNet** with:

Mask:	255.255.255.0
Lower Bound:	192.168.56.2
Upper Bound:	192.168.56.199

Table 2.2: CloudBasicNet

2.2.1. attach all nodes

After this step, modify the network setting. In the setting windows of every node:

- move to *Network* $\rightarrow$ *Adapter 2*;
- enable the network;
- choose *Internal Network*;
- insert the previously created network name.

It is very important activate the *port forwarding* in the nodes to give the possibility to the host to connect to the nodes with ssh.

VM settings $\rightarrow$ *Network* $\rightarrow$ *Advanced* $\rightarrow$ *Port forwarding*

The port forwarding rule for the Master is:

Name $\rightarrow$ *ssh*

Protocol $\rightarrow$ *TCP*

Host IP $\rightarrow$ 127.0.0.1

Host Port $\rightarrow$ 4022

Guest Port $\rightarrow$ 22

I did the same thing for the nodes but with the *Host Port* 3022.
Now all nodes and the local host can communicate via **SSH** protocol.

Code 4: ssh connection from localhost to master

```
ssh -p 4022 user01@localhost
```

2.2.2. master-nodes configuration

By executing `ip link show` we can see the name of the various interfaces:

Code 5: `scp` command

```
ip link show

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue
state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
qdisc fq_codel state UP mode DEFAULT group default qlen
1000
    link/ether 08:00:27:73:b9:25 brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
qdisc fq_codel state UP mode DEFAULT group default qlen
1000
    link/ether 08:00:27:c7:a8:a1 brd ff:ff:ff:ff:ff:ff
```

In this case, is crucial to identify the correct network to use to set the **netplan**. In my case, it is the `enp0s8`. Identifying the right configuration, I modified the following file in this way:

Code 6: `/etc/netplan/50-cloud-init.yaml`

```
network:
version: 2
ethernets:
enp0s3:
dhcp4: true
enp0s8:
dhcp4: false
addresses: [192.168.56.1/24]
```

Modified the file, we need to save it and apply it with this command:

Code 7: new netplan application

```
sudo netplan apply
```

The same instructions must be performed on the client nodes. To have more clarity, I

change the *host name* of the Master Node in **master**, changing the file `/etc/hostname`, and the names of the nodes in `/etc/host` in this way:

Code 8: hosts name change

```
127.0.0.1 localhost
127.0.1.1 master
192.168.56.1 master

192.168.56.2 node02
192.168.56.3 node03
192.168.56.4 node04
192.168.56.5 node05
192.168.56.6 node06
192.168.56.7 node07
192.168.56.8 node08
192.168.56.9 node09

# The following lines are desirable for IPv6 capable
hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

2.2.3. DNSMASQ

To dynamically assign an ip and the ip addresses and hostname to the client nodes on the internal interface, I installed `dnsmasq`. To configure it, modify `/etc/dnsmasq.conf`. The configuration that I used is the following:

Code 9: /etc/dnsmasq.conf

```
interface=enp0s8
bogus-priv
bind-interfaces
cache-size=1000
resolv-file=/etc/resolv.dnsmasq
listen-address=127.0.0.1,192.168.56.1
log-queries
dhcp-range=192.168.56.2,192.168.56.10,12h
```

It is also important to modify /etc/resolv.conf:

Code 10: /etc/resolv.conf

```
nameserver 192.168.56.1
search .
```

we need to link /run/systemd/resolve/resolv.conf with /etc/resolv.dnsmasq with the following command:

Code 11: linking resolv

```
sudo ln -s /run/systemd/resolve/resolv.conf \
/etc/resolv.dnsmasq
```

and finally restart and enable the service with:

Code 12: dnsmasq

```
sudo systemctl restart dnsmasq
sudo systemctl enable dnsmasq
```

A problem that I find is that when you want to enter a node you need to know the name, therefore the ip. This is a trial and error process for the user. Moreover, if you have a DHCP, this process is even more random. So, a good improvement could be to create a script that checks which node is available, maybe trying to start a contact with that IP, and then assign that IP to the node01, and continue like that for all active nodes. In this way, we always have the open nodes in the lowest possible position (node01, node02, etc) and it is easy to access it.

2.2.4. remote access: SSH/SCP

To access nodes, I set the network to use the `ssh` command. This is necessary to communicate between nodes. But a good feature to implement is the key-less access. This process speeds up the test part. The steps to reach this result are:

- create the key in one node with:

Code 13: create the key

```
ssh-keygen
```

- take the public key generated in the `.ssh` directory and transfer it to another node
- add to the `authorized_key` file with these commands:

Code 14: ssh public key

```
mkdir ~/.ssh
chmod 700 ~/.ssh
cat ~/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 644 ~/.ssh/authorized_keys
```

In this way, all the successive times that I need to access a node, the connection became faster and easier.

Another important command that I usually use in this context is the command to communicate information between nodes and localhost, **SCP**:

Code 15: scp command

```
scp -P 4022 /[path]/[to]/[my]/[file] \
user01@localhost:[path]/[to]/[send]/
```

This is pretty useful in the testing phase, due to the fact that I had to transfer the data files to my localhost to be analyzed.

2.3. Shared Filesystem

Another important step that I must take to create my VMs cluster is to create a distributed filesystem accessible from every node in the cluster. The program that I use is NFS.

Code 16: installation of NFS

```
sudo apt install nfs-kernel-server
sudo mkdir /shared
sudo chmod 777 /shared
```

In the following, I edited this file:

Code 17: /etc/exports

```
...
/shared/ 192.168.56.0/255.255.255.0(rw,sync,
no_root_squash,no_subtree_check)
...
```

I restart the service and create two shared subdirectories:

Code 18: /etc/exports

```
sudo systemctl enable nfs-kernel-server
sudo systemctl restart nfs-kernel-server
sudo mkdir /shared/data /shared/home
sudo chmod 777 /shared/data /shared/home
```

These steps are very useful for testing and evaluating cluster performance.

2.4. Useful Scripts

In the end, to speed up my testing process, I decided to create two scripts to open and close the entire cluster from the terminal. This is because they are common actions.

Code 19: upVM.sh

```
#!/bin/bash
echo "Power On of the VM cluster..."
VBoxManage startvm "Ubuntu_VM_Master" --type headless
VBoxManage startvm "Ubuntu_VM1" --type headless
VBoxManage startvm "Ubuntu_VM2" --type headless
echo "Power On command send successfully to all VMs."
```

Code 20: downVM.sh

```
#!/bin/bash
echo "Shut down of the VM cluster..."
VBoxManage controlvm "Ubuntu_VM_Master" acpipowerbutton
VBoxManage controlvm "Ubuntu_VM1" acpipowerbutton
VBoxManage controlvm "Ubuntu_VM2" acpipowerbutton
echo "Shutdown command send successfully to all VMs."
```

3. Containers Setup

Like VMs, Containers are fundamental for modern software industry. Provides a safe and controlled environment to run your code, with no hardware virtualization and high performance, similar to the native one.

The use cases are analogous to and have the same versatility as VMs.

I remind the reader that I implement the container in a machine with Ubuntu as OS, this could change the installation procedure and the tests results much. All materials that I used came from online guides, videos and a university lecture.

You could find all the configuration on my GitHub: github.com/GabrieleGranzotto/Docker-Configuration.

3.1. Create Containers Structure

There are many programs that provide the possibility to create a container. I used Docker. There is by far the most popular container program. As first thing, I had to decide the structure of my project, which disposition my data should have, and make some crucial design choice. I decide for:

Code 21: Docker project structure

```
Docker-Configuration/  
|  
+--docker-compose.yml  
|  
+--master/  
|  |  
|  +-- Dockerfile  
+--nodes/  
|  |  
|  +-- Dockerfile  
+--shared_data/
```

Finished this part, I need to configure each single file. This has been a trial and error

process: many times I modified a file and then I needed to re-modify to adjust or add some feature.

3.1.1. docker compose

To create the cluster in Docker is virtually mandatory to use the feature `docker compose`. To set the cluster behavior, it is used the `docker-compose.yml`. In the file, I insert the various services that i want to build:

- master
- node01
- node02

The real configuration of the service is not in the `docker-compose.yml` file indeed, it is in the specific Dockerfile that I show later. In my specific case, I have two Dockerfiles, one for the master and one for the other nodes.

In my case, I decided to create two subdirectories where Dockerfiles is located. This forced me to specify the pattern in the `docker-compose.yml` writing in `build` the `context: ..`, that means "in this directory" (the `docker-compose.yml` directory), and `dockerfile: [directory]/Dockerfile`, that is the actual path starting from `..`

We can even see `networks`, that is an internal virtual network that can bind all the nodes one another. The command to build the cluster image is:

Code 22: build docker cluster image

```
docker compose build
```

This has been done in the directory where the `docker-compose.yml` file is. If the user wants to build an image from scratch, with no caching, the command is:

Code 23: build the image but with no cached data

```
docker compose build --no-cache
```

To turn on the cluster, the command is as follows:

Code 24: turn on the cluster

```
docker compose up -d
```

and to turn off, the command is:

Code 25: turn off the cluster

```
docker compose down
```

These commands are everything I needed to control my cluster.

Code 26: docker-compose.yml

```
services:
  master:
    build:
      context: .
      dockerfile: master/Dockerfile
    container_name: master
    hostname: master
    networks:
      - testnet
  node01:
    build:
      context: .
      dockerfile: nodes/Dockerfile
    container_name: node01
    hostname: node01
    networks:
      - testnet
  node02:
    build:
      context: .
      dockerfile: nodes/Dockerfile
    container_name: node02
    hostname: node02
    networks:
      - testnet
networks:
  testnet:
```

3.1.2. shared filesystem

Code 22 is just the backbone of the file.

To create a shared filesystem, I had to have a few more lines.

This adds to the file is simply the creation of a virtual shared storage called `/data` that are

linked to a real directory in the actual directory on the host machine called `shared_data`.

Code 27: `docker-compose.yml`

```
services:
  master:
    ...
  volumes:
    - ./shared_data:/data

  node01:
    ...
  volumes:
    - ./shared_data:/data

  node02:
    ...
  volumes:
    - ./shared_data:/data
...
volumes:
  ./shared_data:
```

3.1.3. resource limits

The last thing — for now — to implement is the resource limits. In the VM is mandatory writing the maximum resources, in docker is not.

So I had to specify it to give, on paper, the same computational power and memory to VM and Docker. So I set `cpus: '2.0'` and `mem_limit: 2G`.

It is important to specify that *in theory* the resources are the same, but *in practice* the resources are very different:

- in the VM, the machine dedicates two segregated cores;
- in Docker the maximum usage of the CPU cannot exceed the 200% of a core-power, so it is a dynamic allocation.

It is similar for the RAM:

- in VM, we have a part in the memory totally dedicated of VM's processes;
- in Docker, all container's processes are mixed in the local host process memory. So, even the memory is allocated dynamically.

Code 28: docker-compose.yml

```
services:
  master:
    ...
    cpus: '2.0'
    mem_limit: 2G

  node01:
    ...
    cpus: '2.0'
    mem_limit: 2G

  node02:
    ...
    cpus: '2.0'
    mem_limit: 2G
  ...
volumes:
  ./shared_data:
```

3.2. Configuration Files

The creation of `docker-compose.yml` is not sufficient to create the cluster. Naturally, it is necessary to describe the container that you want to build.

In my case I have two types of container, so I wrote two `Dockerfile`.

The `Dockerfile` is the file that describes all the characteristic that the container has to have.

3.2.1. master Dockerfile

The master `Dockerfile` is composed of different parts:

- the OS used (in my case the same of the VM) and some environment settings;

Code 29: OS and ENV

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive \
    TZ=Europe/Rome
```

- all the installed packages;

Code 30: apt commands

```
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    nano vim curl wget build-essential \
    software-properties-common \
    hpcc mpich stress-ng sysbench iozone3 iperf3 netcat-
    openbsd \
    openssh-client \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
```

- the creation of the ssh key;

Code 31: ssh-keygen

```
RUN ssh-keygen -f /root/.ssh/id_rsa -N "" \
    && echo "Host *" > /root/.ssh/config \
    && echo "    StrictHostKeyChecking no" >> /root/.ssh/
config \
    && echo "    UserKnownHostsFile /dev/null" >> /root/.
ssh/config \
    && chmod 600 /root/.ssh/config
```

- the working directory and the command the master must execute (bash)

Code 32: working directory and CMD

```
WORKDIR /root/
CMD ["/bin/bash"]
```

3.2.2. nodes Dockerfile

The Dockerfile of the node differs just in a few things:

- there are some commands that allow to use SSH from master to node, this because linux denies access to the root user by default;

Code 33: some ssh command

```
RUN mkdir /var/run/sshd \  
    && echo 'root:root' | chpasswd \  
    && sed -i 's/#PermitRootLogin prohibit-password/  
PermitRootLogin yes/' /etc/ssh/sshd_config \  
    && sed -i 's/#PasswordAuthentication yes/  
PasswordAuthentication yes/' /etc/ssh/sshd_config \  

```

- when the node turns on it starts with the `sshd` service on to consent the master to log in.

Code 34: node CMD

```
CMD ["/usr/sbin/sshd -D"]
```

With these settings, the cluster is ready to test the performance.

3.3. SSH problems & strategy

Before I start talking about performance, it is important to talk about some troubles that I have to face with the ssh keyless process.

The core problem is how to move the public key from the master node to the other nodes. The simplest answer is to generate all the key in the local machine and then move them to the cluster nodes. But this creates two problems:

- the process is not automated
- the process is not safe because we move the private key that is important to stay in the same place where the user created it.

A strategy that I found is to use a feature that Docker provides that is `healthcheck`: it is a test that Docker Compose does until the test passes.

Code 35: healthcheck

```

services:
  master:
    ...
  healthcheck:
    test: ["CMD", "test", "-f", "/data/id_rsa.pub"]
    interval: 1s
    timeout: 60s
    start_period: 3s
    ...
  node01:
    ...
    depends_on:
      master:
        condition: service_healthy
    ...
  node01:
    ...
    depends_on:
      master:
        condition: service_healthy
    ...

```

The test is simply "if the public key exists in the shared directory". If the test is "affirmative", then the other nodes start.

When all nodes are on, the two children copy the content of the public key from the shared directory to `.ssh/authorized_keys`. All this process is done to avoid *race condition* between nodes.

I had to find a solution for another problem: the nodes have to copy the public key in runtime, so I inserted a `setup.sh` script into **CMD**.

Code 36: node CMD

```

CMD ["/bin/bash", "-c", "/root/setup.sh && /usr/sbin/sshd -D"
]

```

The script is as follows:

Code 37: script.sh

```
#!/bin/bash
mkdir -p /root/.ssh
chmod 700 /root/.ssh
touch /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
cat /data/id_rsa.pub >> /root/.ssh/authorized_keys
```

Another security problem is that the password of the root user is set in clear, not encrypted, as root. So I decide to change the password to a less vulnerable choice:

Code 38: password client

```
...
RUN mkdir /var/run/sshd \
    && echo 'root:\$(openssl rand -base64 12)' | chpasswd \
    ...
...
```

The node generates a random encrypted password. This is not a perfect choice because no one knows the value of the password, but it is better than the alternative.

3.4. Terminal in a Docker Machine

The last command is used to enter a node and use the `bash` program of the node, but the syntax is analogous to that of every other program:

Code 39: docker-compose.yml

```
docker exec -it node bash
```

4. Performance Tests

In this chapter, I present the results of these tests. At the end of this chapter my goal is to present the data that can show the differences between VMs performances and containers performances.

On a theoretical basis, the Virtual Machine performs worse in virtually every test because it is a segregated machine that communicates with the hardware through a virtualized layer. This adding step increases the time and the communication overflow, and consequently reduces the performances.

On the practical side, the results can be very different, for many reasons. This is why I had to test and analyze the results.

4.1. Methodology & Applied Tests

The tests that I did are always on the same machine, so with the same hardware. Containers and VMs have the same amount of RAM, CPU cores and the same OS. This is made to not invalidate all the collected data.

The details of the system are reported below.

CPU:	Intel Core i5-8365U 1.60GHz
RAM:	16GB DDR4
DISK:	256GB SSD NVMe
LOCAL HOST OS:	Ubuntu 24.04
CORE USED:	2 cores
RAM USED:	2GB
TEST MACHINE OS:	Ubuntu 24.04

Table 4.1: Test Machines Detailed

The tests that I did test processing power, memory, disk performances (writing and reading) and network communication.

It is important to note that almost all tests were performed to test the single node and cluster. To test the cluster performance, I use **mpirun**, which provides to spread the same test in all nodes of the cluster.

The tests that I used are the following:

- **HPL** (processing power);
- **FFT** (processing power);
- **starDGEMM** (processing power);
- **stress-ng** (processing power);
- **sysbench** (processing power);
- **MPI random access** (memory);
- **starSTREAM triad** (memory);
- **stress-ng** (memory);
- **sysbench** (memory);
- **iozone** (disk performance);
- **iperf3** (network communication).

4.1.1. mpirun

mpirun is the launcher for Message Passing Interface (MPI) applications. It executes a program in parallel across multiple processors or different nodes in a cluster, handling the communication environment between them.

To use it, the command is the following:

Code 40: Quick Usage Guide

```
mpirun -np [number_of_processes] \  
--hostfile [filename] ./executable
```

4.1.2. HPCC

HPCC is a program that collects a number of tests for the evaluation of computing power and memory.

The most famous and used is **HPL**, which solves a dense linear system ($Ax = b$) to measure double-precision floating-point performance. It is the standard benchmark used to rank supercomputers in the **top500** list.

Another test is **FFT**, which computes a 1 dimension Discrete Fourier Transform. It measures floating-point performance but heavily stresses the memory subsystem and network due to complex, non-local data access patterns.

StarDGEMM is a CPU-bound benchmark measuring Double-precision General Matrix Multiplication (DGEMM). It tests the raw calculation power and efficiency of the floating-point units.

StarSTREAM Triad measures sustainable memory bandwidth using the Triad operation ($a[i] = b[i] + q \times c[i]$). It tests how fast data can move between the memory and the processor.

MPI random access measures the rate of integer updates to random locations in distributed memory. Specifically, it evaluates the memory latency and performance of the network interconnect between nodes.

The command that I use to test both infrastructures are the following:

Code 41: HPCC command

```
# to evaluate single node performance
mpirun -np 2 hpcc
# to evaluate the entire cluster
mpirun -np 6 --hostfile ./myhost.txt hpcc
```

This will run all the tests in *HPCC*. To run, this program needs a configuration file, *hpccinf.txt*, in the current directory.

And for the cluster test a file for the *host configuration*.

Code 42: myhost.txt

```
master slots=2
node01 slots=2
node02 slots=2
```

I decide to test the entire structure with 3 different NB values (128, 192, 256) and 2 N values (6000, 10000).

In the end, the test creates an output file *hpccoutf.txt* with all the results.

Code 43: hpccinf.txt

```

HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out          output file name (if any)
6                device out (6=stdout,7=stderr,file)
1                # of problems sizes (N)
6000            Ns
3                # of NBs
128 192 256     NBs
0                PMAP process mapping (0=Row-,1=Column-major)
1                # of process grids (P x Q)
2                Ps
3                Qs
16.0            threshold
1                # of panel fact
2                PFACTs (0=left, 1=Crout, 2=Right)
1                # of recursive stopping criterium
4                NBMINs (>= 1)
1                # of panels in recursion
2                NDIVs
1                # of recursive panel fact.
1                RFACTs (0=left, 1=Crout, 2=Right)
1                # of broadcast
1                BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1                # of lookahead depth
1                DEPTHs (>=0)
2                SWAP (0=bin-exch,1=long,2=mix)
64              swapping threshold
0                L1 in (0=transposed,1=no-transposed) form
0                U  in (0=transposed,1=no-transposed) form
1                Equilibration (0=no,1=yes)
8                memory alignment in double (> 0)

```

4.1.3. stress-ng

Stress-ng is a stress-test tool designed to load the system. The **CPU** stressors run complex math to overheat/exhaust cores, while **Memory** stressors aggressively allocate and trash RAM to find hardware faults. The commands that I use are the following:

Code 44: stress-ng

```
# single node test
stress-ng --cpu 2 --timeout 60s
stress-ng --vm 1 -vm-bytes 1G --timeout 60s
# cluster test
mpirun -np 3 --hostfile ./myhost.txt stress-ng --cpu 2 --
timeout 60s
mpirun -np 3 --hostfile ./myhost.txt stress-ng --vm 1 -vm
-bytes 1G --timeout 60s
```

I repeated this test 3 times for different timeouts to track the difference: 60s, 120s, 180s.

4.1.4. sysbench

Sysbench is a scriptable multi-threaded benchmark. The **CPU** test verifies speed by calculating prime numbers, while the **Memory** test measures throughput via sequential or random read/write operations.

The commands to use sysbench are:

Code 45: sysbench commands

```
sysbench cpu --cpu-max-prime=10000 run
sysbench memory --memory-total-size=1G run
```

I run this commands for different `cpu-max-prime` (10000, 50000, 100000, 150000) and `memory-total-size` (1G, 2G, 10G, 20G, 40G, 60G). This to see the evolution of the values.

4.1.5. iperf3

Iperf3 is a network tool that generates traffic to measure the maximum achievable bandwidth. Test throughput, packet loss, and jitter between a client and a server using TCP or UDP.

The commands to use iperf3 are:

Code 46: sysbench commands

```
# on server
iperf3 -s
# on client
iperf3 -c node01
```

4.1.6. iozone

Iozone is a filesystem benchmark that performs a wide variety of file operations (read, write, re-read, stride). It analyzes disk I/O performance across different file sizes and block sizes. To test iozone I used different commands with different parameters:

Code 47: sysbench commands

```
# no direct I/O, with caching
iozone -a -g 4G -i 0 -i 2 -b [results_file]
# direct I/O, without caching
iozone -a -g 4G -i 0 -i 2 -I -b [results_file]
```

Now I describe the parameters used:

- **-a**: automatic settings;
- **-g**: maximum size of a single file, it is important that is more than RAM dimension;
- **-i**: type of data to collect (I collect write, rewrite, random write and random read);
- **-I**: the program uses direct I/O, trying to bypass the cache;
- **-b**: name of the `results_file` in xls extension.

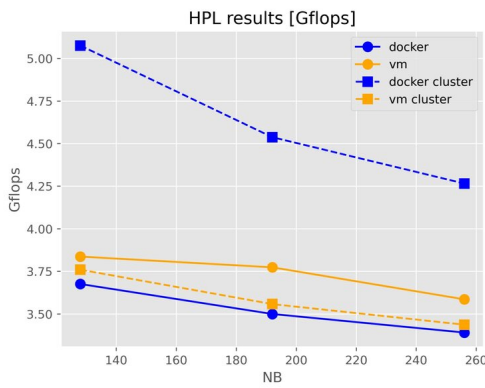
All this test is repeated in the **local machine directory** and the **shared directory** to check the different performances.

4.2. Results: Processes

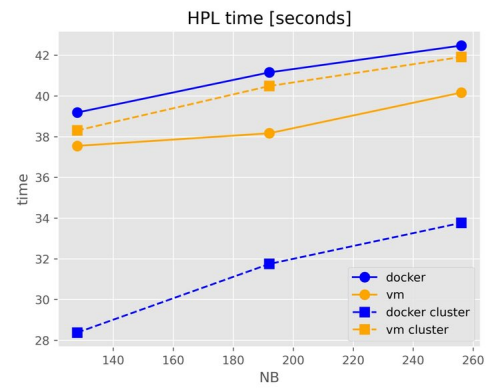
In this section, I analyze the results of the tests that stress processors. The main test for this part is HPL, but there are also interesting tests that give us useful information.

4.2.1. HPL

I run HPL for three different NBs and, as I expect, the performance reduces and the time increases, for all tests: docker with $N=6000$, docker with $N=10000$, VM with $N=6000$ and VM with $N=10000$, for single node and for the cluster. In Figure 4.1, we can see that the performance of the four problems tested is proportionally inverse to the time, as we expected. The highest result belongs to the **docker cluster**, followed by the **VM single node** and then the **VM cluster**, the lowest is for the docker single node. The reason why this happened is because, for the single node case, the VM is an isolated structure, with dedicated resources. This characteristic makes it that the VM single core outperforms docker single core structure. The Docker cluster has better communication efficiency between nodes, which is why Docker outperforms the VM cluster. Even the single node VM outperforms the cluster one, because the problem is too small to compensate the large amount of communication overhead.



(a) HPL computational results for different architectures and technology

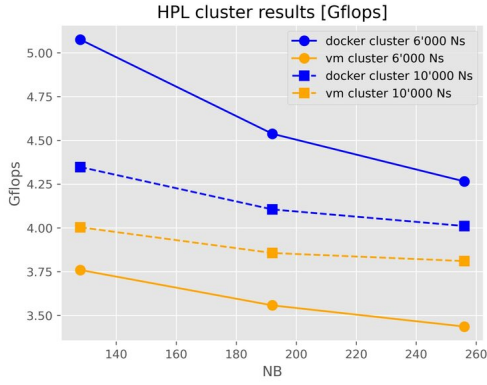


(b) HPL execution times for different architectures and technology

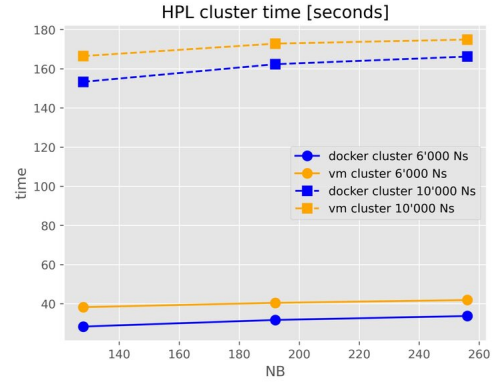
Figure 4.1: HPL results that compares single node and cluster for VM and for docker: time (a) and processing power (b).

We can see the same effect described before in figure 4.2. Here, the graph shows the performance of the VM cluster and the Docker cluster, for $N=6000$ problem and $N=10000$ problem. For bigger problem, Docker performs worse than smaller one, while for VM is the opposite.

In summary, if the problem is large enough, the cluster always performs better, even if it is implemented with a virtual machine. If the problem is small, the cluster of VM is lacking because of communication overhead. Instead, the VM single core performs better than the Docker single core because the dedicated resources strongly increase the performance. This is a problem, again, just for small problems. For large problems, this problem disappears.



(a) HPL computational results in a cluster



(b) HPL execution times in a cluster

Figure 4.2: HPL results for multiple nodes, $N=6000$ vs $N=10000$: time (a) and processing power (b).

4.2.2. FFT

In this test we can see the same behavior of HPL. Docker cluster performs slightly better than all the other configurations, while VM single node outperforms Docker single node and VM cluster.

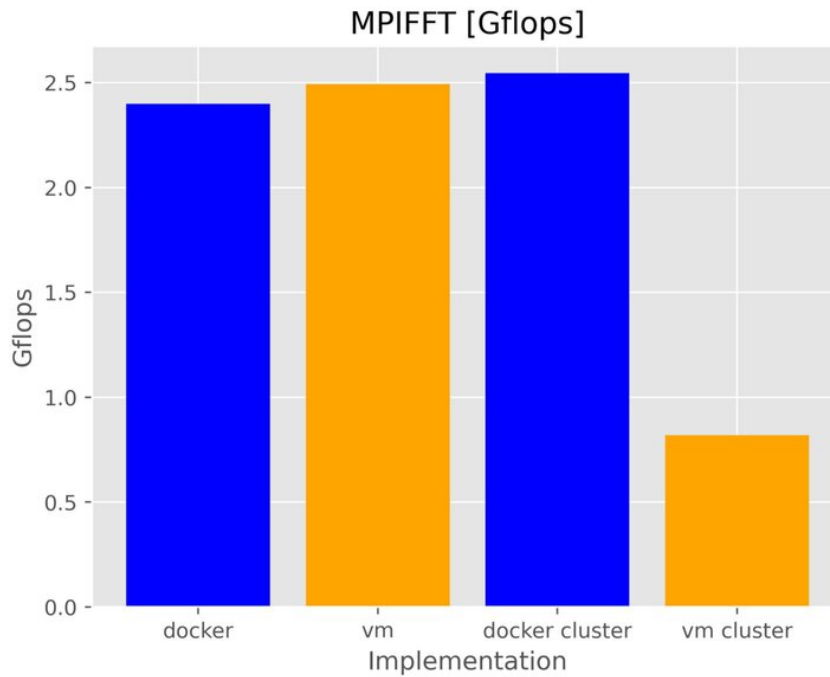


Figure 4.3: MPIFFT performance test

The results are shown in Figure 4.3. The VM cluster performs more than 60% less than VM single node.

For the other three configurations, the performance is very similar. This should be because FFT test is not as demanding as HPL so the test cannot have a big performance difference.

4.2.3. starDGEMM

This test is very CPU intensive, and the results are really surprising.

In Figure 4.4, we can see that the single node configurations perform more than double the cluster configurations. The answer that I can give is that the multiplication of matrices was too small to be efficiently parallelized. This leads to a large communication overhead that penalized the cluster configurations.

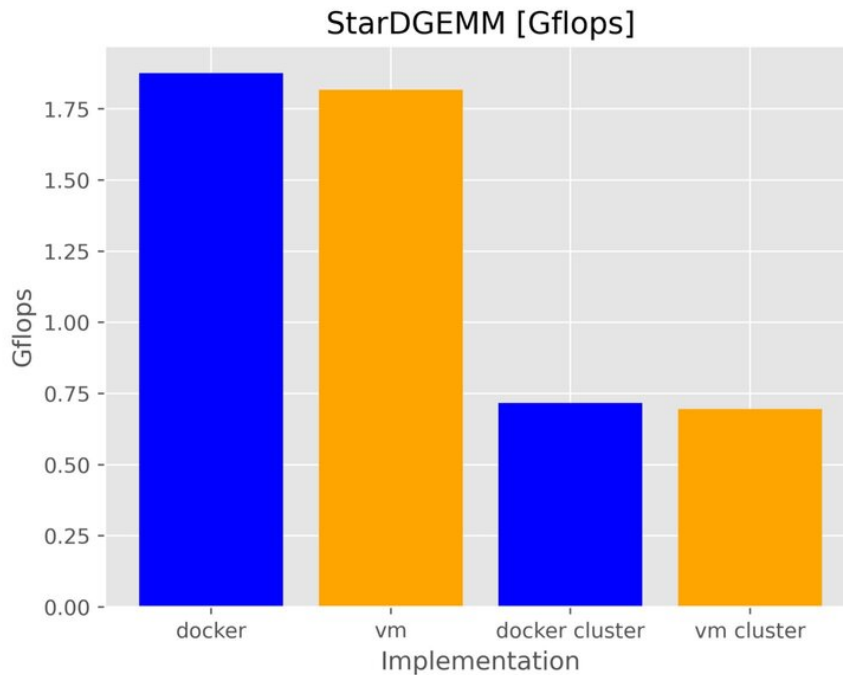


Figure 4.4: StarDGEMM performance test

4.2.4. stress-ng

In this test we can see that the performance of Docker and VM are very similar. I run this test for different timeouts (60s, 120s, 180s) and the results are almost the same. So we can conclude that both technologies are able to provide the same CPU performance for this kind of test and it's not affected by the timeout.

A thing to note is that for both technologies, we can see a node that performs more than twice the other two. I think this effect is due to the CPU architecture and the way the OS allocates the resources.

4.2.5. sysbench

In this test we can see that Docker and VM have almost the same performance. The results are very similar for all the different `prime numbers`.

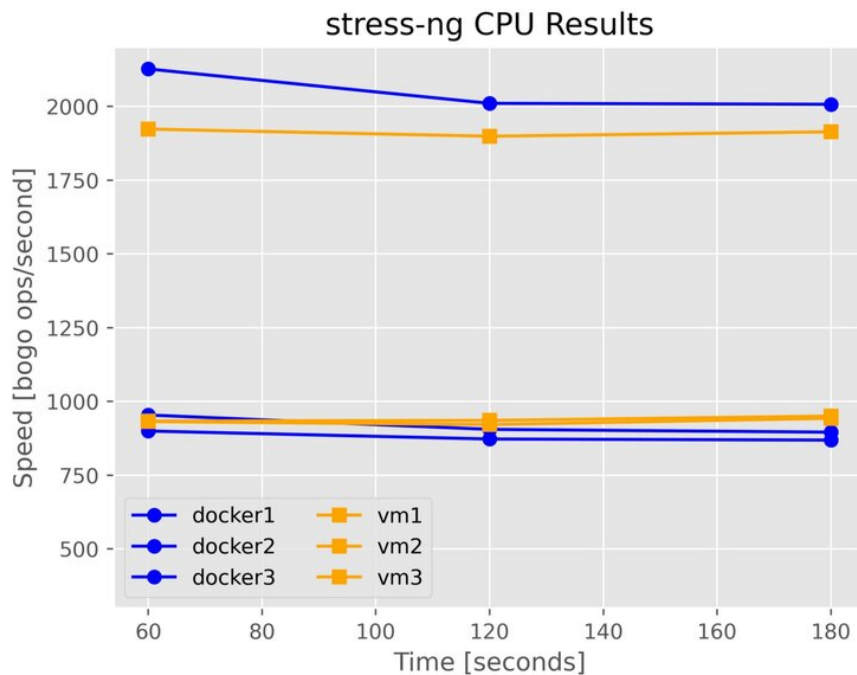


Figure 4.5: stress-ng: CPU performance test

This result is reasonable because the allocated resources are the same and in this case the virtualization overhead is negligible.

4.3. Results: Memory

In this section, I analyze the results of the tests that stress memory. The memory in VM is very different compared to Docker, because in VM the memory is allocated and dedicated, while in Docker the memory is shared with the host OS.

This characteristic theoretically slows down the VM memory performance.

4.3.1. MPI random access

Even in this case we can see the outperformance of single node configurations over cluster ones. Docker single node performs slightly better than VM single node, while Docker cluster performs better than VM cluster.

Obviously, the communication overhead penalizes the cluster configurations. In a cluster environment, the memory access is slower because of the network latency.

4.3.2. starSTREAM triad

We can analyze the same behavior of previous test in Figure 4.8. The single node configurations outperform the cluster ones, and Docker performs slightly better than VM

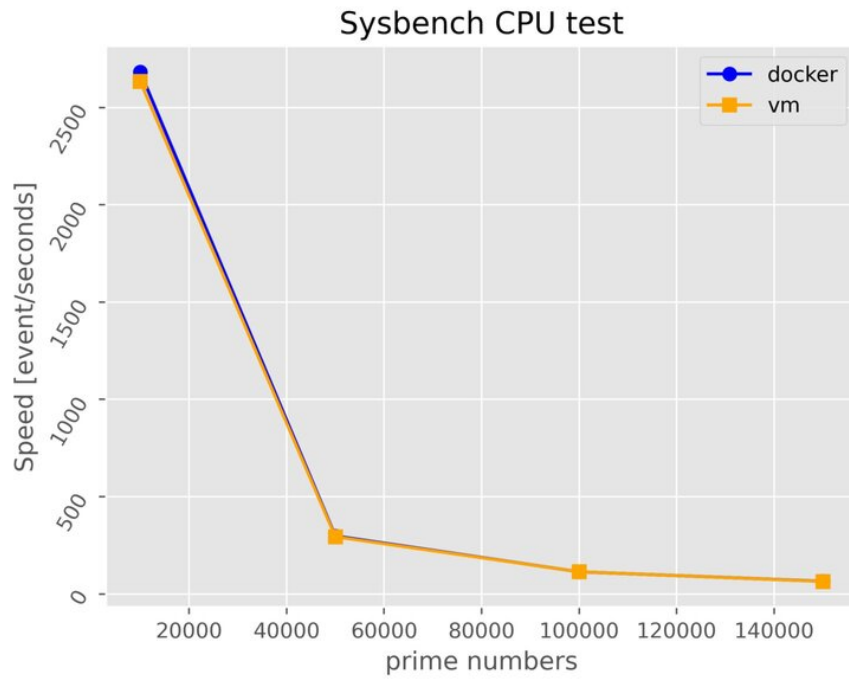


Figure 4.6: sysbench: CPU performance test

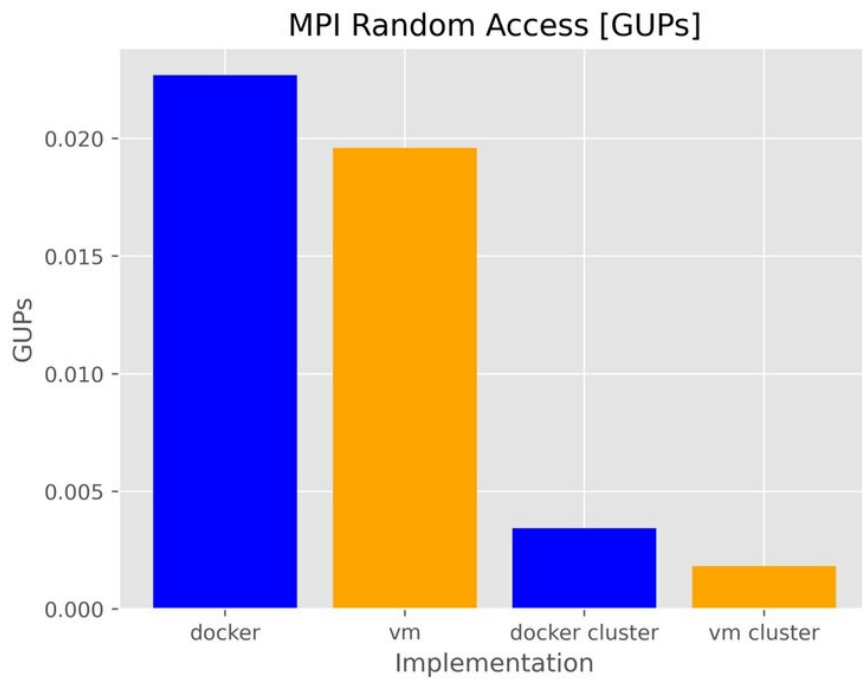


Figure 4.7: MPI Random Access performance

in both configurations.

In this test, more than the previous one, the difference between Docker and VM is not so marked. I think because this test is more focused on memory bandwidth rather than latency.

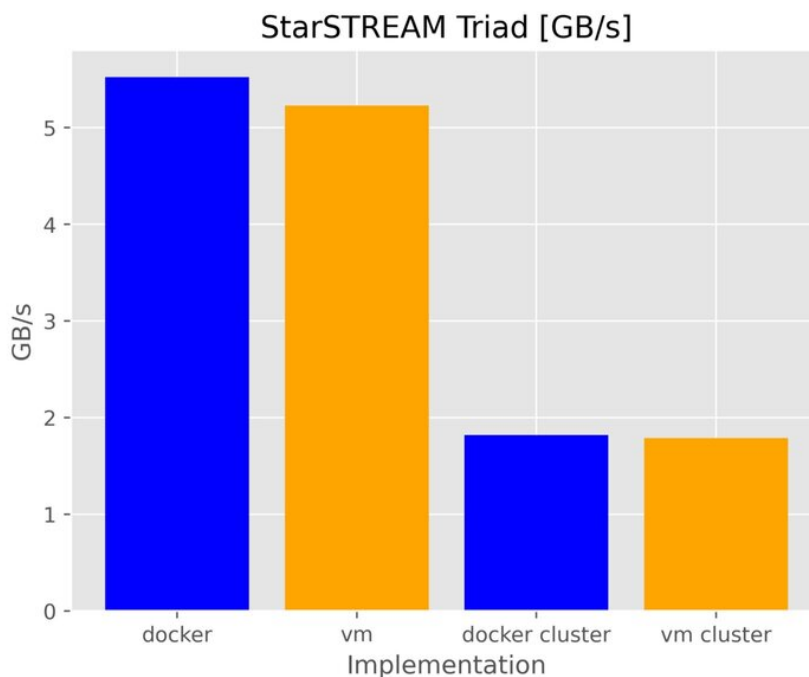


Figure 4.8: StarSTREAM Triad performance

4.3.3. stress-ng

In this test, that is the memory version of stress-ng, we can see a similar behavior of the CPU version. Docker has better performance than VM almost always.

All the considerations made for the CPU version about cores are valid also for this memory version.

4.3.4. sysbench

This test are the same of the CPU version, but focused on memory, but the results are very different.

The performance of Docker is noticeably better than VM. This is because the memory allocation in VM is dedicated and fixed, while in Docker the memory is shared with the host OS, making the access faster.

I need to repeat this test for different **memory sizes** to see when the data became stable, with no more big changes.

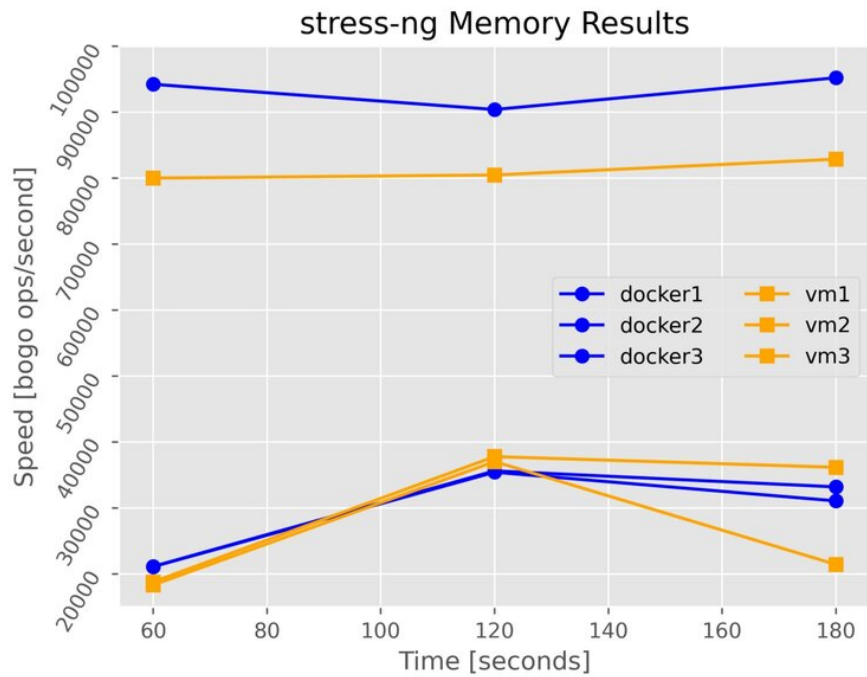


Figure 4.9: stress-ng memory test performance

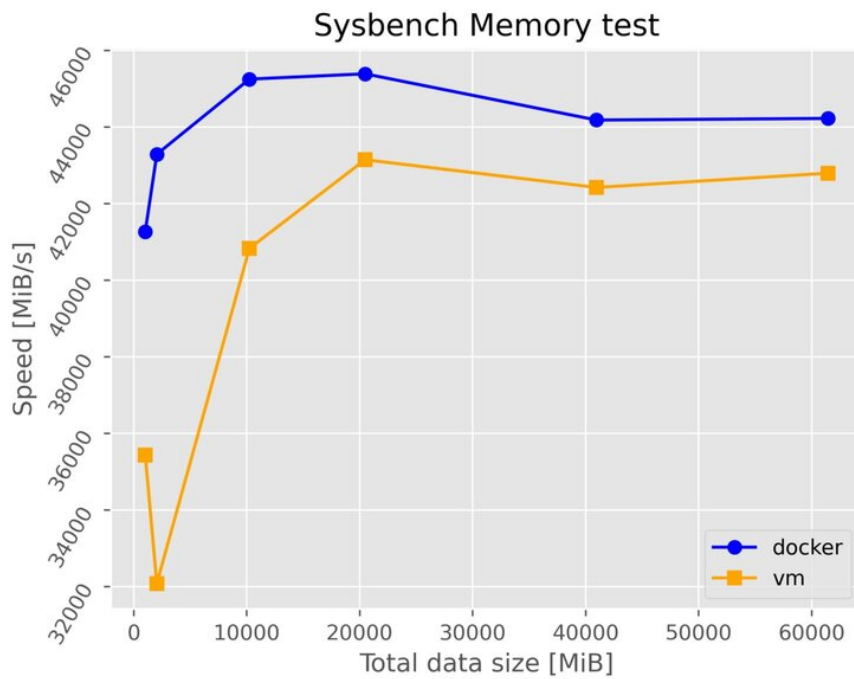


Figure 4.10: sysbench memory test performance

4.4. Results: Disk

In this section I will analyze the results of the disk performance tests.

For disk performance test, the main tool used is `iozone`. I decide to test with and without direct I/O, that is a way to bypass the cache and write/read directly on the disk.

In this way it emerges how the caching system can influence the performances.

4.4.1. `iozone`: no direct I/O

I start with the results without direct I/O, so with caching system enabled. I tested both in local directory and shared directory.

Theoretically, the shared directory is slower because of the network overhead.

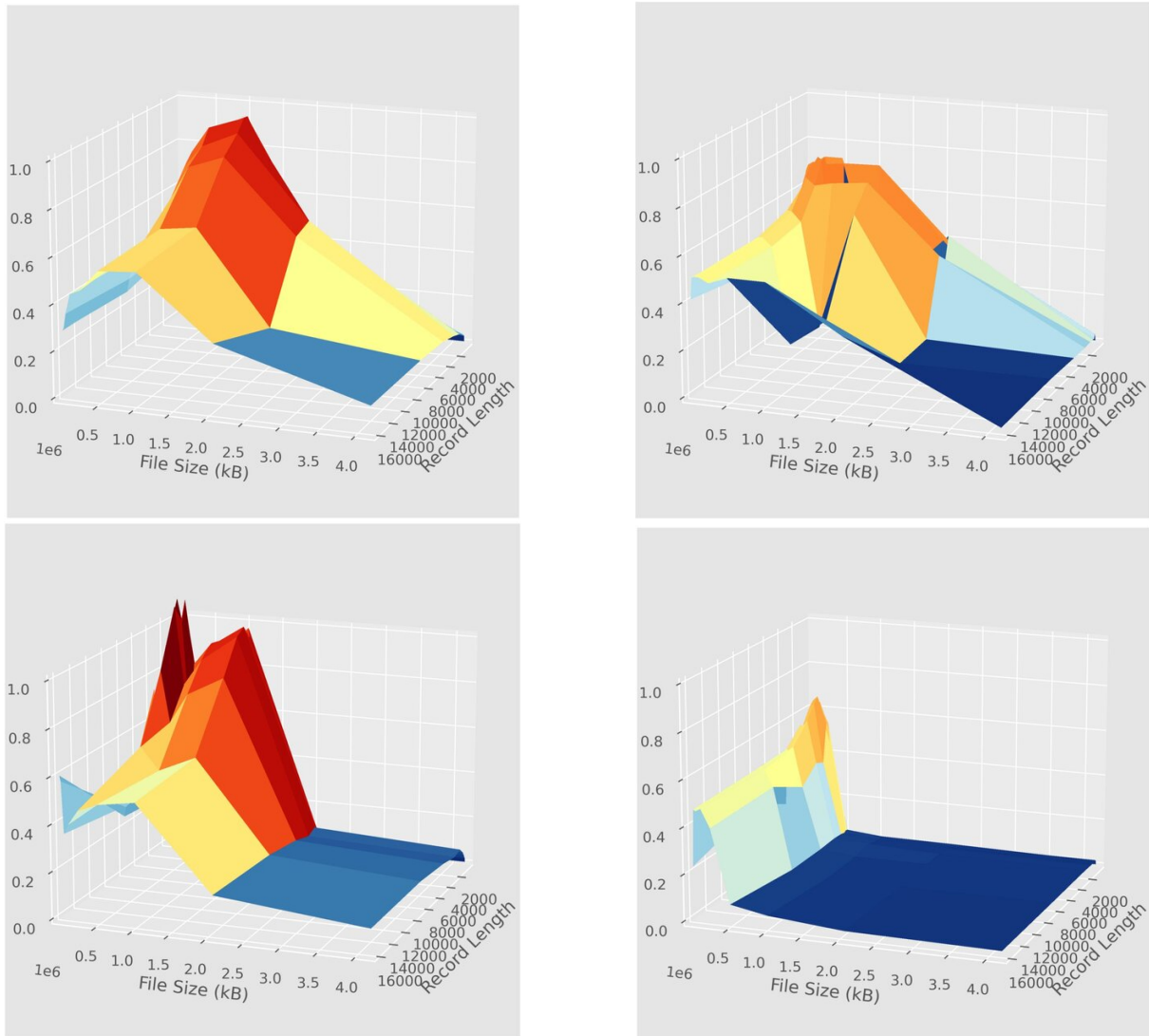


Figure 4.11: `iozone` local directory performance with no direct I/O: (a) random read (docker), (b) random write (docker), (c) random read (vm) (d) random write (vm)

In figure 4.11 we have four plots: in the left side the docker results, in the right side

the VM results, in the first row the random read results, in the second row the random write results.

It is clear that Docker performs decisively better than VM in both read and write operations. This effect is due to the fact that VM use a virtualized disk that slow down the operations. I suppose that, with caching system enabled, the difference is more marked because the cache system is more efficient in Docker than in VM.

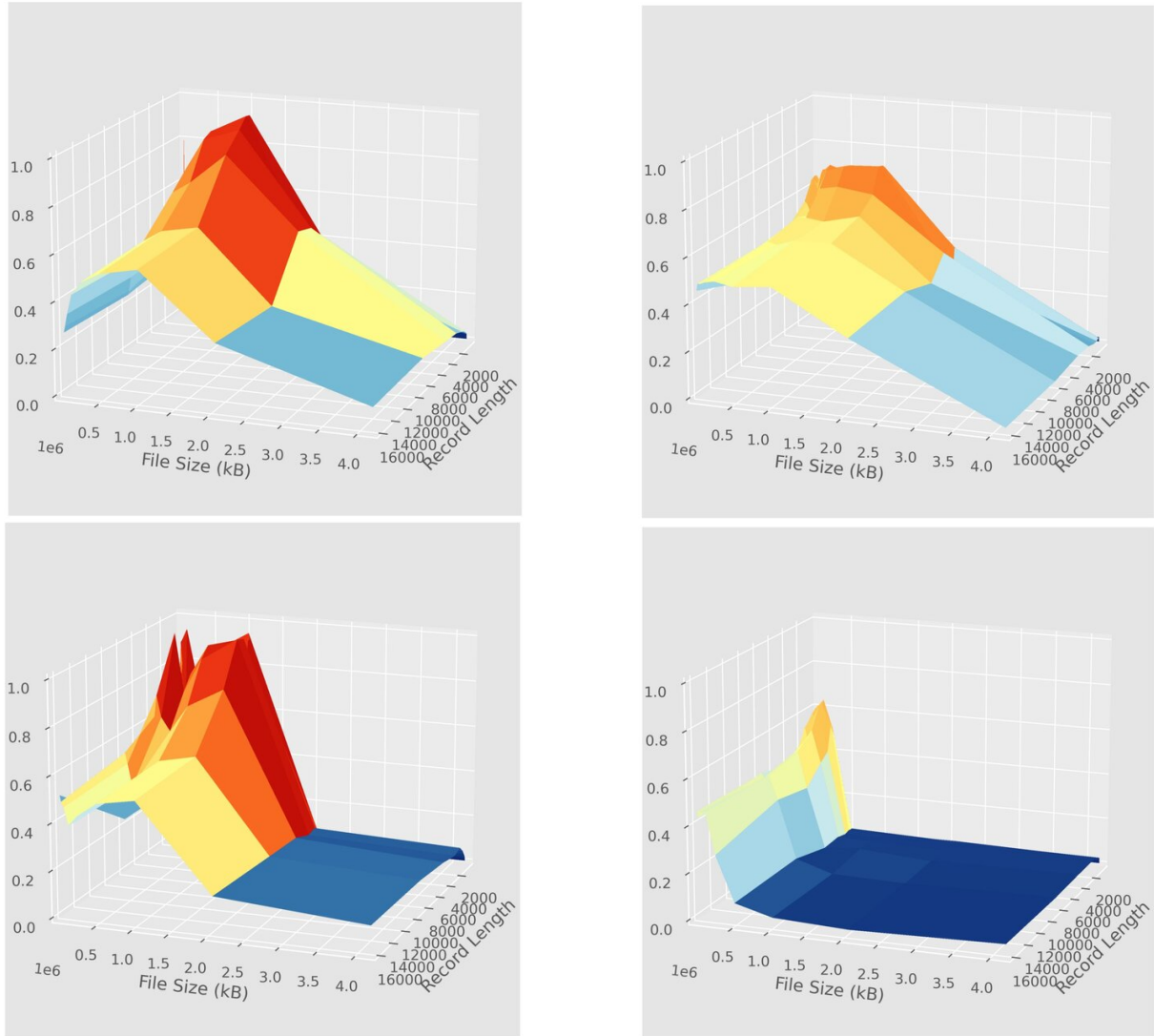


Figure 4.12: iozone shared directory performance with no direct I/O: (a) random read (docker), (b) random write (docker), (c) random read (vm) (d) random write (vm)

In the shared directory case, shown in figure 4.12, the results are overall similar. The differences, compared to local test, are negligible.

4.4.2. iozone: direct I/O

Iozone test with direct I/O, so without caching system, gives us interesting results. First of all, I set the same scale as the previous test to have a better comparison.

The results, shown in figure 4.13, are really different from the previous test.

The similarities between the four previous plots are that Docker always performs better than VM.

The graphs show a decrease in performance of about one order of magnitude compared to the previous test with caching system. This prove us how important is the caching system for disk performance.

In a qualitative way, not quantitative, the behavior is similar to the previous test.

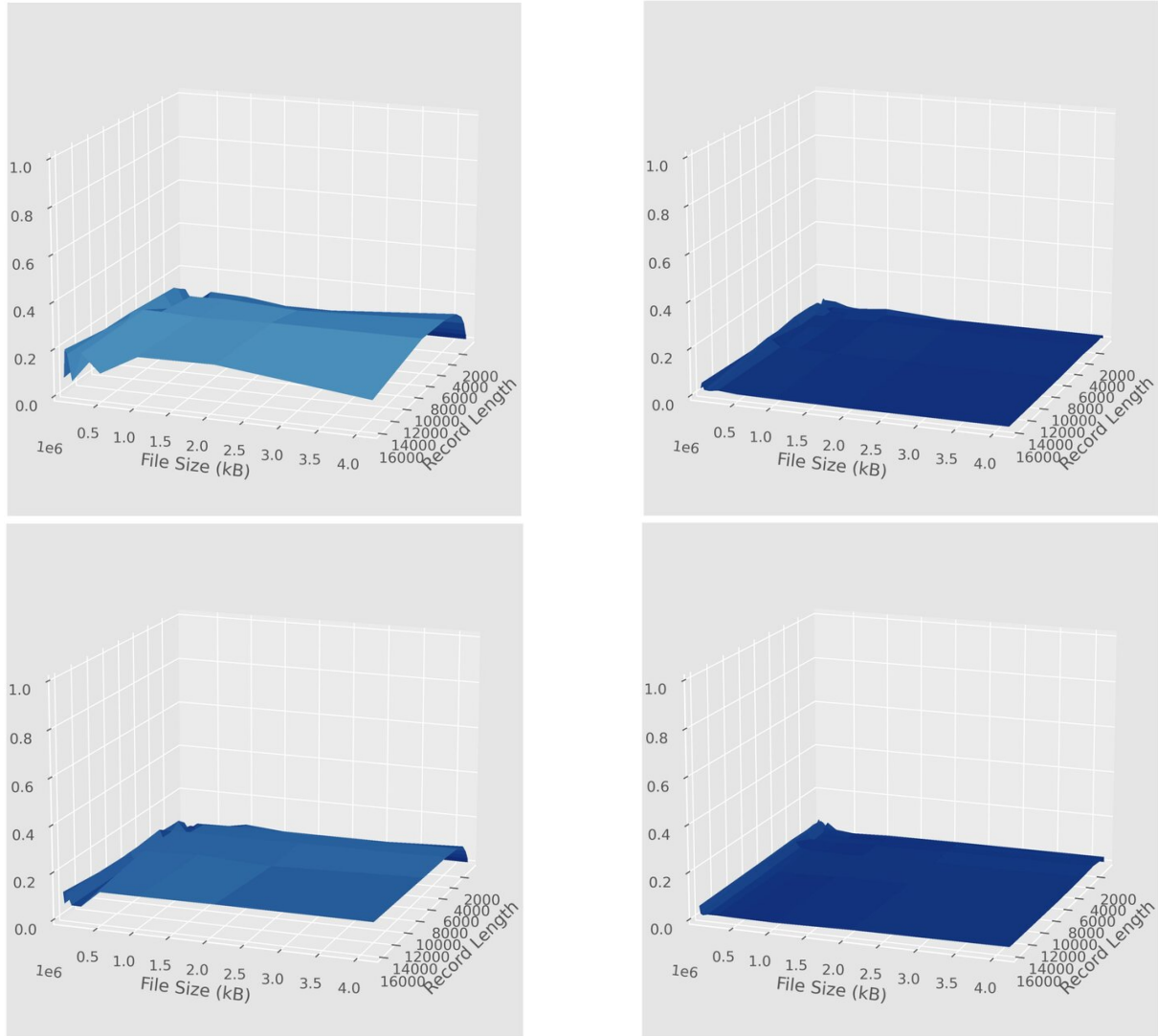


Figure 4.13: iozone local directory performance with direct I/O: (a) random read (docker), (b) random write (docker), (c) random read (vm) (d) random write (vm)

In figure 4.14, the results are practically identical both in qualitative and quantitative way.

The network overhead does not influence so much the performances in this case.

At the end, we can conclude that Docker has better disk performance than VM, and this because the **hypervisor** layer in VM slow down the disk operations.

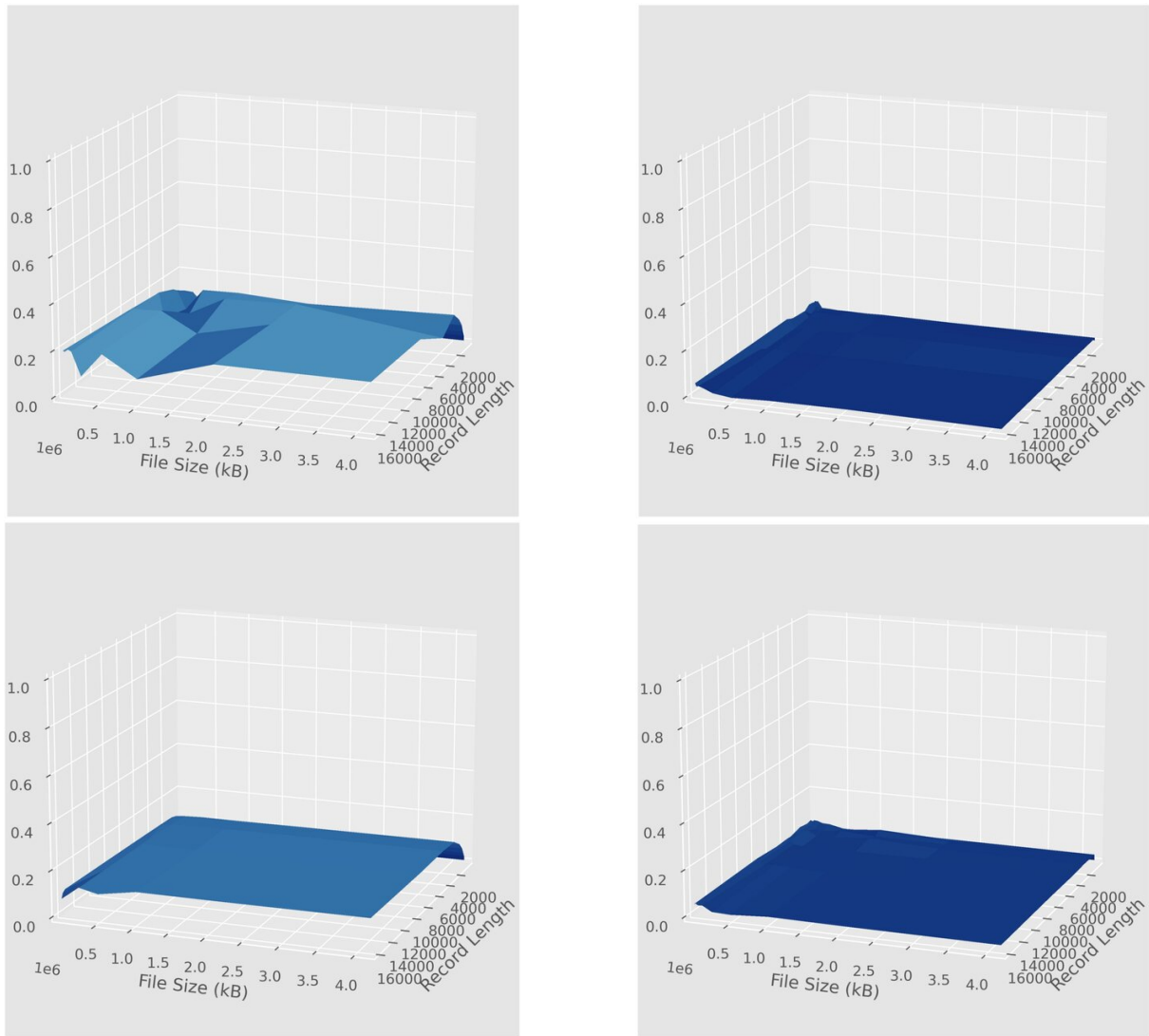


Figure 4.14: iozone shared directory performance with direct I/O: (a) random read (docker), (b) random write (docker), (c) random read (vm) (d) random write (vm)

4.5. Results: Communication

4.5.1. iperf3

As last test, I analyze the network communication performance with iperf3. The result, shown in figure 4.15, does not leave space for doubts.

Docker outperforms VMs by a large margin. This is because the network virtualization in VM adds a large overhead that strongly penalizes the communication performance.

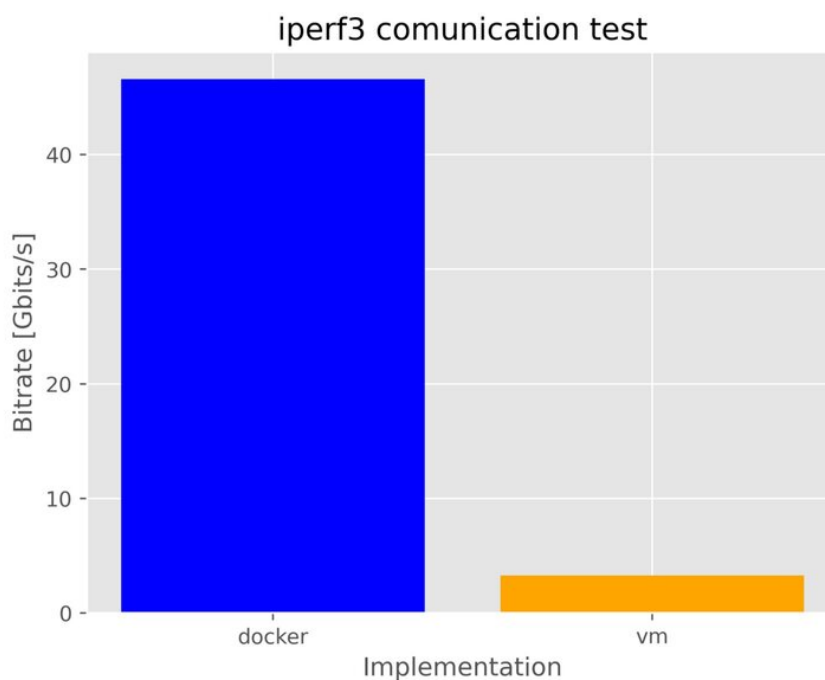


Figure 4.15: iperf3 communication performance

The difference is more than an order of magnitude, making Docker in strong advantage for network communication services.

The test was done using TCP protocol, that is the most used in network communication. The two involved nodes are in the same physical machine, so the results should be even more marked.

In the previous tests, often the explanation of the difference between Docker and VM could be attributed to the inefficient communication between nodes in VM.

Here, this effect is evident and quantifiable.

5. Conclusions

In this report, we have explored the fundamental concepts and practical applications of VM and Docker containerization technology. I began by discussing the architecture of Virtual Machines (VMs) and I then examined the process of creating and managing VMs, highlighting best practices for optimizing performance and security. I then briefly discussed Docker, including its components such as Docker Engine, Docker Images, and Docker Containers. I then delved into the process of creating and managing Docker images, highlighting best practices for building efficient and secure images.

Finally, I compared VMs and Docker containers, discussing their respective advantages and disadvantages, and provided guidance on when to use each technology based on specific use cases and requirements.

Overall, this report told that both VMs and Docker containers are powerful tools for managing and deploying applications, but the performance and efficiency of Docker containers make them a more attractive option for many use cases.

Personally, I found Docker even easier to use and manage compared to VMs. The program installation and configuration process was straightforward, the rapidity of deployment was impressive, and the portability in other systems made it a versatile choice for various applications.

Nonetheless, VMs still have their place in certain scenarios, such as when running legacy applications or when a higher level of isolation is required.

In conclusion, the choice between VMs and Docker containers ultimately depends on the specific needs and requirements of the application being deployed, but it is comprehensible that containerization technology is becoming increasingly popular in modern software development and deployment practices.